

Featured in Diamondhead

Intern Article →



PORTFOLIO · 2026

Rian Kawahara

Software Engineer / Solutions Engineer



Hello Friends! W!th LINEヤフー 2026

Featured in @lycorp_jp Story →



Meet Lumo! β版発表会

製品デモ 2025.12.11 Tokyo →

📍 Tokyo, Japan

MONASH University

B.S. Computer Science

Monash University →

Tokyo, JP · rian.dev.official@gmail.com · GitHub · LinkedIn · riankawahara.github.io

WORK · MAR 2025 – PRESENT

Software Engineer / Solutions Engineer · Little Help Agency LLC

Full-stack and customer-facing engineer for Little Help Connect and Lumo.cx — building and deploying AI-native messaging × CRM integration platforms for enterprise clients across travel, retail, and education.

React | Vite | TypeScript | Kotlin | Spring WebFlux | Node.js | GCP (Cloud Run, Pub/Sub) | PostgreSQL | HubSpot | LINE



Lumo booth — venue overview, LINE Yahoo Hello Friends 2026

HIGHLIGHTS

- Designed and implemented LINE Group → HubSpot Company sync, adopted by 10+ enterprise clients (incl. JTB Asia Pacific, KOMEHYO) — turning messaging platforms into structured CRM input.
- Among the main frontend contributors on Lumo.cx in the first 8 months (1000+ commits, 300+ PRs across React / Vite / TypeScript), with growing backend contributions in Kotlin / Spring WebFlux.
- Rewired the mobile app's API endpoint mapping to integrate with the Lumo backend and owned the deployment of the rebadged mobile app to all Lumo users.
- Represented Lumo as bronze sponsor at LINE Yahoo Hello Friends 2026, engaging ~20 prospects in product demos and bilingual technical Q&A at the booth — ~10 qualified leads over two days.
- Covered technical support during a period of support-team absence (alongside core engineering duties) — resolved 10+ customer tickets solo via discovery calls + diagnostic queries against GCP / HubSpot logs, including unblocking a partner agency that was using LHC to configure LINE Official Account workflows for their end client (a long-standing critical issue had stalled their delivery).
- Handled enterprise clients' security check sheets (セキュリティチェックシート) for LHC and Lumo — translating system architecture into accurate JP-language answers on data storage, encryption, access control, auth flows, and third-party dependencies to satisfy each client's internal security review.
- Solo-delivering a CSV data ingestion system (custom column mapping from external candidate / ATS data into LINE-driven follow-ups) and a new property management surface for a major enterprise PoC with multi-agency Japan deployment — leading customer kickoff.
- Delivered Meet Lumo! product demo to 30+ stakeholders, a tailored Little Help Connect demo to One Education Group (education-agency client, engagement contributed to closing the deal), technical support for Hububble's Taiwan rollout, and bilingual cross-border representation with Klaviyo and JTB AI-agent PoCs.
- Built an internal Slack → Devin → Jira automation that turns bug mentions into tracked Jira tickets — improving project-management hygiene around bug capture and triage.

WORK · APR – AUG 2024

Software Engineer Intern · MEJORA Corporation

Reporting directly to the CEO in a 4-person team. Built generative-AI demo products that the CEO used as the centrepiece of AI-DX consulting pitches to Japanese enterprises.

React | Firebase | Node.js / Express | OpenAI Chat Completions | LangChain (explored) | Python | Selenium | Hugging Face | pandas | Figma



Final rendered survey — interactive output from a Japanese free-text prompt

HIGHLIGHTS

- Led the implementation of the AI Survey Builder PoC end-to-end (React + Firebase + OpenAI API) — used by the CEO in customer meetings as a clickable conversation-starter for AI-DX engagements.
- Proposed product directions based on market research scrapers I built (Selenium + Hugging Face) and analysed in pandas — moved beyond pure implementer to product-shaping.
- Researched candidate tech stacks and produced Figma mockups before committing to implementation, keeping iteration cost low.
- Prototyped a LangChain-based variant in Jupyter; kept the shipped path on direct API after concluding LangChain wasn't earning its dependency at the PoC stage.

WORK · DEC 2023 – FEB 2024

Software Engineer Intern · Diamondhead Corporation

Worked on the Feature Improvement Team (~10 members) on a centralised e-commerce management SaaS handling client products across Amazon, ZOZOTOWN, and other platforms.

HTML | CSS | JavaScript | Django | PostgreSQL



Featured in Diamondhead's intern article (note.com)

HIGHLIGHTS

- Improved the word count feature to display accurate counts across two different input modes — a small UX detail with disproportionate visibility on the product's authoring surface.
- Replaced the user account modification flow that relied on stale cached data with an immediate-update version — fixing a class of confusing post-edit display bugs.
- Redesigned the product image position editing mode: changed the default behaviour and implemented adjustable settings configurable to each client's preferences.
- Wrote test specifications for shipped features so colleagues could test them, reviewed peers' test specs, and verified code prior to release.
- After project completion, presented progress and feature summaries to the team.

Education, Skills & Credentials

A snapshot of the formal training, tech surface area, and recognised credentials behind the work in this portfolio.

TECH STACK

Frontend	React · Vite · TypeScript · JavaScript · Next.js · HTML5 · CSS3
Backend	Kotlin · Spring WebFlux · Python · Node.js / Express · Django
AI / LLM	OpenAI API · LangChain (explored) · LlamaIndex (exposure) · Prompt design · RAG patterns
Data & Infrastructure	PostgreSQL · Firebase · Oracle SQL · Docker · GCP (Cloud Run, Pub/Sub) · Azure Web App (exposure)
Platforms & Integrations	HubSpot · LINE · Klaviyo
Data Science	R · pandas · Jupyter · Vega-Lite · Tableau · scikit / tree models · Selenium
Mobile / Design	Swift · Figma
Languages	English (Native) · Japanese (Native / Bilingual)

EDUCATION

Bachelor of Computer Science · Monash University

2020 – 2024

Started in the Science faculty (incl. data-science coursework — Business Intelligence & Data Warehousing, Data Analytics, Modelling for Data Analysis, Data Visualisation), transferred into Computer Science.

High School Diploma · St. Mary's International School

Graduated 2019

Tokyo · English-language K–12 program.

LICENSES & CERTIFICATIONS

HubSpot Sales Hub Software Certified

HubSpot Academy · May 2026

HubSpot Inbound Certified

HubSpot Academy · May 2026 · Expires Jun 2028

Klaviyo Practitioner Certificate

Klaviyo · May 2026

LINE Official Account Basic

LY Corporation · Jul 2025

AWARDS

🏆 **Business Award Winner**

Tokyo Open Data Hackathon 2024 (Tokyo Metropolitan Government Digital Services Bureau) · Oct 2024

Carbon credit acquisition support service using generative AI.

Devin CVE Loop — event-driven autonomous-agent remediation

An end-to-end demo I built using the Devin API: scan a forked Apache Superset for known CVEs, spin up Devin sessions per finding, let it open remediation PRs back to the repo, and surface 'is this actually working?' signals — task status, success/failure, throughput — to an engineering leader.

Self-driven project · Devin API · June 2026

Python | Devin API | Docker | GitHub (forked Apache Superset) | Event-driven workflow | Webhooks | Observability / reporting

WHAT I BUILT

- Event trigger: CVE-style issues created on the forked Superset repo become the input event. Each issue describes a vulnerable package + a target version, and triggers the orchestration layer.
- Devin orchestration (Python): builds a structured prompt per finding, starts a Devin session via the Devin API, tracks the session, and follows it through to PR creation linked back to the originating issue.
- Observability surface answering the brief's exact question — 'if I were an engineering leader, how would I know this is working?' Active vs completed sessions, success/failure signals, throughput across issues. Dockerised for reproducible runs.
- Captured run: 5 CVE issues across 5 packages remediated, 6 PRs opened (flask 2.3.3 → 3.1.3, pyjwt 2.12.0 → 2.13.0 ×2, jaraco-context 6.0.1 → 6.1.0, starlette 0.49.1 → 1.0.1).

WHAT I LEARNED

- Translating an ambiguous brief into a working end-to-end system was the first deliverable; picking 'CVE remediation' and framing it for an engineering-leader audience preceded any code.
- Treating Devin as a first-class system actor (orchestrated programmatically, tracked, reported on) — not as a one-shot helper — was the design decision that made everything else fall into place.
- Observability scoped at the same time as the orchestration logic, not bolted on. For an autonomous-agent system, that's the difference between 'sometimes works' and 'shippable to a customer.'
- The 5-min Loom doubles as a representative sample of how I demo a technical product to a senior stakeholder — concrete, on a real codebase, no fluff.

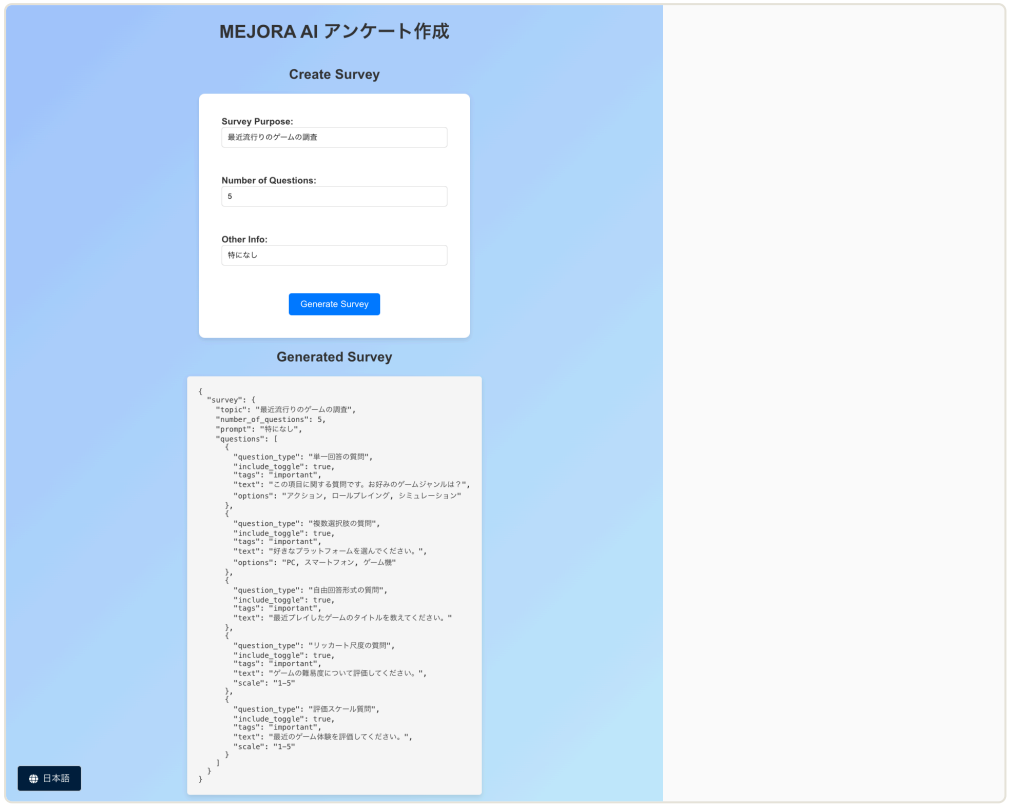
▶ 5-min demo (Loom) · loom.com/share/9cf5448f58fc4a399fb7b796427227ef · Solution repo · github.com/riankawahara/devin-cve-loop · Forked Apache Superset · github.com/riankawahara/superset

MEJORA AI Survey Builder — a customer-facing AI PoC

A working generative-AI prototype built during my engineering internship at MEJORA, used by the CEO as the centrepiece of AI-DX consulting pitches — turning 'we can help you with generative AI' into a clickable artefact a non-technical executive could try in 30 seconds.

MEJORA Corporation · Software Engineer Intern · Apr–Aug 2024 · Reporting to the CEO in a 4-person team

React | Firebase | Node.js / Express | OpenAI Chat Completions | LangChain (explored) | Python | Selenium | Hugging Face | pandas | Figma



Earlier iteration — input + raw JSON

WHAT I BUILT

- Built an end-to-end PoC that turns a free-text Japanese prompt (e.g. 'スマホの調査をしたい') into a structured JSON survey spec via OpenAI Chat Completions, then renders it as a live interactive survey on the same page — all in one view.
- Backend on Node/Express with prompt design constraining the LLM output to a typed schema (multiple-choice, scale, open-text). Prototyped a LangChain variant in Jupyter but kept the shipped path on direct API for PoC simplicity.
- Led the demo-product implementation, proposed product direction based on market research scraping (Selenium + Hugging Face + pandas), built Figma mockups, and researched stacks before committing to implementation.

WHAT I LEARNED

- The demo IS the deliverable. The point was a 30-second conversation-starter for AI-DX consulting, not a shipping survey product. Forward-deployed work has the same shape: artefact in service of the customer conversation, not the other way around.
- Designing for the non-technical eye is a separate skill from building the system. The CEO needed to walk a marketing director through a clickable form, not a CTO through a JSON schema.
- When your differentiator lives upstream, don't build a worse version of an existing utility downstream — I proposed Google Forms API integration so MEJORA wouldn't own the response-collection surface.

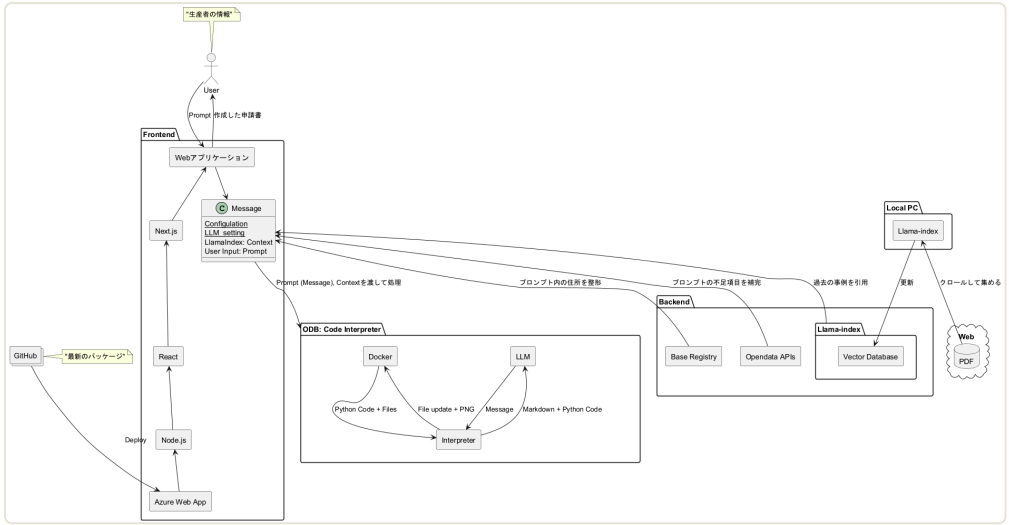
Survey app on GitHub · github.com/riankawahara/mejora_survey_app · Market research scrapers · github.com/riankawahara/MEJORA_WEBSCRAPE

CO2TechAssist — Carbon Credit Acquisition with Generative AI

Business Award winner at the 2024 Tokyo Governor's Cup Open Data Hackathon. A vertical application of dx-junkyard's Open Data Bridge (ODB) platform — applying the 'recipe' mechanism (stored prompts + outputs) to carbon-credit request forms so past applications can be reused to draft new ones.

Tokyo Metropolitan Government Digital Services Bureau · Hackathon (Oct 2024) · dx-junkyard / CO2TechAssist sub-team

Next.js · React · Node.js · TypeScript · Python · LLM (Code Interpreter) · LlamaIndex · Vector DB · Open Data APIs · Docker · Azure Web App



ODB system architecture (dx-junkyard, PlantUML)

WHAT I BUILT

- ODB (the underlying platform, built by dx-junkyard pre-hackathon) has four planes: a Next.js frontend, a Docker-sandboxed Code Interpreter (LLM writes Python, Docker executes it), a backend wrapping Tokyo's Base Registry + Open Data APIs, and a LlamaIndex-backed vector store of crawled precedent PDFs.
- CO2TechAssist is the vertical application: past J-Credit / blue-carbon / municipal-forest-absorption applications become 'recipes' the system reuses to pre-fill a new carbon-credit application faster than starting from a blank form.

WHAT I LEARNED

- Being precise about my role: I was not a core builder. I joined CO2TechAssist after my first sub-team (DX-ReBridge) didn't advance, then participated in stand-ups, discussions, and onboarding for the post-hackathon rollout phase before the team disbanded — so I did not ship code into the system. What I did get was inside exposure to how a small civic-tech team forms, pitches, and operates.
- The 'recipe' abstraction is the interesting idea: a generic prompt+output substrate becomes leverage when paired with a sharp domain. Civic open data + an LLM grounded in known-good past applications is a pattern worth reaching for again.
- Civic-tech framing matters even from the audience side. Watching how the team translated architecture into something a municipal stakeholder would act on is the same translation skill customer-facing engineering needs.

Open Badge — Business Award · openbadge-global.com · ODB Frontend (dx-junkyard) · github.com/dx-junkyard/opendata-bridge-frontend

AFL Match Predictor & Analytics Dashboard

Final-year Data Science capstone — an end-to-end data product turning raw AFL match feeds into a predictive model and an interactive BI dashboard for fans and analysts.

Monash University · FIT3163 + FIT3164 (DS Capstone) · 2024

React | TypeScript | Firebase | Cloud Functions | Chart.js | AG Grid | TanStack Table | Python (model) | R (analysis) | Jupyter | Docker | Fly.io

WHAT I BUILT

- Data ingestion: CSV pipelines (Papa Parse + csvtojson) normalising per-match, per-team, and per-player feeds into a consistent Firestore schema.
- Predictive model in Python notebooks (feature engineering + training), with supporting statistical analysis in R.
- Interactive BI dashboard in React + TypeScript: sortable/filterable data grids (AG Grid, TanStack Table), team and player drilldowns, and time-series visualisations of form, ladder position, and prediction confidence (Chart.js).

WHAT I LEARNED

- Most lessons came from data cleaning, not the model. Inconsistent column naming, mid-season schema changes, and missing fixture values drove iteration cost — the training loop was the easy part.
- A model output is more credible when the UI shows the supporting numbers next to it. Adding form, home/away splits, and head-to-head history changed how testers reacted to the same prediction.
- Polyglot pipelines (Python → JSON → React) are fine for a capstone but the seams are where bugs hide. Push schema enforcement closer to ingest, not later.

Source on GitHub · github.com/rian-uni-material/fit3164_react

Japanese Prefecture Happiness — Data Visualisation Atlas

A multi-chart Vega-Lite analysis of the 2020 Japanese Prefecture Happiness Ranking — exploring which factors (income, weather, crime, welfare, typhoons) correlate with how happy each of Japan's 47 prefectures actually feels.

Monash University · FIT3179 Data Visualisation · 2023

Vega-Lite v5 | D3 | TopoJSON | JavaScript | Tableau Public

WHAT I BUILT

- Joined the 2020 Happiness Ranking with multiple prefecture-level CSVs (income, minimum wage, welfare uptake, temperature, typhoon hits 1951–2022, crime, never-married %) on a common Prefecture key, then wired the data into a TopoJSON of Japanese prefectures via Vega-Lite's lookup transform — one source of truth per chart, no duplication.
- Picked encodings per question: diverging colour for temperature, sequential greens for welfare uptake, magnitude-encoded bubbles for income overlaid on the region map. Tooltips on every chart let viewers read underlying numbers, not just the colour.
- Companion piece: a Tableau dashboard analysing airline passenger satisfaction — different stack, same underlying skill (pick the right encoding to expose the relationship being asked about).

WHAT I LEARNED

- Encoding > prettiness. The biggest readability lift came from choosing diverging vs sequential vs bivariate scales appropriately — not from polishing typography or background art.
- One source of truth per chart. Vega-Lite's lookup against a single TopoJSON kept all eight charts aligned — when I tried per-chart denormalised data, every fix had to be made in eight places.
- Tooltip-first design was the single highest-impact interaction — it turned a static essay into something testers clicked around in.

Source on GitHub · github.com/rian-uni-material/FIT3179_Data_Visualisation_II

Equipment-Hire Data Warehouse — FIT3003 Major Assignment

A semester-long pair project turning a messy operational OLTP database for an equipment-hire business into a clean, queryable star/snowflake data warehouse — including ER analysis, two competing schema designs, data-quality remediation, and SCD strategy.

Monash University · FIT3003 Business Intelligence & Data Warehousing · Sem 2 2023 · Pair project

Oracle SQL | ER modelling | Star schema | Snowflake schema | SCD Types 1–6 | Dimensional modelling

WHAT I BUILT

- Conceptual design: reverse-engineered the operational E/R, then designed two competing star/snowflake schemas. The two differ in normalisation depth (CustomerType pulled out as its own dimension in V2, Equipment + Category snowflaked, Staff added as a tracked dimension) — making the trade-off between query simplicity and dimension reuse explicit.
- Data quality: surfaced and remediated real source errors — duplicate customers, negative QUANTITY values, START_DATE after END_DATE, hire totals miscalculated against (End − Start) × UnitHirePrice × Quantity, dates set in 2090, null CATEGORY descriptions.
- Implementation: wrote the full DDL + INSERT/SELECT to materialise both versions in Oracle. V1 produced 8 tables; V2 reorganised into another 8 with CustomerDIM/StaffDIM/EquipmentDIM split out and CategoryDIM snowflaked off Equipment.

WHAT I LEARNED

- Data quality is half the job. Surfacing duplicates and miscalculated totals is a microcosm of every real-world data integration — the warehouse is only as good as your willingness to clean what comes upstream.
- Dimensional choices are downstream choices. Picking SCD Type 2 vs Type 1 changes what queries are even possible a year later. Justifying each call against expected query patterns is a habit worth keeping.
- Two designs beat one. Being forced to build two schemas made the trade-offs visible — a 'design two, pick one, document why' move I'd reach for again.

Phishing Website Detection — FIT3152 Data Analytics (R)

An individual data-analytics project in R: build, compare, and tune classifiers to predict whether a website is a phishing site or legitimate, using the PhiUSIIL phishing dataset from UCI.

Monash University · FIT3152 Data Analytics · Sem 1 2024 · Individual

R | Decision Tree | Naïve Bayes | Bagging | Boosting | Random Forest | ANN | ROC / AUC | Cross-validation | R Markdown

WHAT I BUILT

- Six baseline classifiers (Decision Tree, Naïve Bayes, Bagging, Boosting, Random Forest), all evaluated under the same train/test split via confusion matrix, accuracy, ROC and AUC plotted on a shared axis.
- Variable-importance analysis across the tree models to identify which predictors matter and which could be dropped. Derived a deliberately-shallow 'hand-classifiable' tree small enough that a human could follow the diagram.
- Tuned the best tree-family classifier via hyperparameter adjustment + cross-validation, plus an ANN with normalisation and one-hot encoding, plus one extension model (a non-course R package) — all compared in the same evaluation table.

WHAT I LEARNED

- Comparable evaluation is the deliverable. The artefact that mattered wasn't any single classifier — it was the table putting accuracy / AUC / ROC for all eight models side-by-side under identical conditions.
- Simple models still earn their keep. The hand-classifiable shallow tree wasn't the most accurate but was the only one where I could point at a node and explain the decision out loud — interpretable baselines often beat slightly-better black boxes.
- Variable importance is feedback to the data pipeline. Dropping zero-importance attributes upstream is cheaper than carrying them through, and overlap across tree models is a more durable signal than any one model's choices.

PhiUSIIL dataset (UCI) · archive.ics.uci.edu/dataset/967

Boston Housing Regression Analysis — FIT2086 Modelling for Data Analysis

A statistics-led modelling assignment: fit and prune a multiple linear regression on Boston housing data in R, with explicit handling of multiple-comparison correction, stepwise model selection, prediction intervals, and what the coefficients actually mean for a council trying to raise median house value.

Monash University · FIT2086 Modelling for Data Analysis · Sem 2 2023 · Individual

R | lm() multiple linear regression | Bonferroni correction | Stepwise (BIC, both directions) | 95% confidence intervals | Interaction effects

WHAT I BUILT

- Fit the full model with lm(medv ~ ., data = housing) → $R^2 = 0.7089$ (adjusted 0.6942) on 237 d.f. Applied Bonferroni correction ($\alpha = 0.05/12 \approx 0.0042$) — only chas, rm, dis, ptratio, lstat survived, much tighter than naive $p<0.05$.
- Stepwise BIC selection (both directions). Final pruned equation: E[medv] = 29.19 + 4.60·chas – 17.38·nox + 4.82·rm – 0.94·dis – 0.96·ptratio – 0.49·lstat.
- Translated coefficient signs into council-actionable advice (Charles River frontage ↑, average rooms ↑, NOx ↓, distance ↓, pupil-teacher ratio ↓). Predicted a new suburb at $21,919.60 with 95% CI ($20,762.10, $23,077.10).

WHAT I LEARNED

- Always report a prediction with an interval, never a point estimate alone. The 95% CI tells a council something fundamentally different from a single number — customers act on the difference.
- Bonferroni-adjust before celebrating a 'significant' result. Pre-correction, 9 predictors looked associated; post-correction, only 5. The difference is 'we found something' vs 'we found noise.'
- Coefficient signs are the deliverable for non-statistical stakeholders. No one outside the team cares about BIC stepwise — they care that 'cutting pupil-teacher ratio raises predicted home value.'